Kalpa Plus membership investigation

All data is synthetic. Follow DATA_DICTIONARY.md. Set DATA_DIR to your downloaded folder. You can load the CSVs from local storage or mounted Google Drive.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
from pathlib import Path
import pandas as pd
DATA_DIR = Path("/content/drive/My Drive/IIT-Gn_FirstPresentation")
AS_OF = pd.Timestamp("2026-10-09 23:59:59")
members = pd.read_csv(DATA_DIR / "members.csv", parse_dates=["join_date"])
renewals = pd.read_csv(DATA_DIR / "renewal_opportunities.csv", parse_dates=["expiry_date", "renewal_deadline"])
events = pd.read_csv(DATA_DIR / "member_events.csv", parse_dates=["event_time"])
print(members.shape, renewals.shape, events.shape)
```

```
(8580, 8) (11580, 7) (152057, 10)
```

Before coding

What is one row in each table? Which key is unique? Which join can multiply rows? State the denominator for renewal rate. Why does an October expiry need special treatment?

```
members.head()
```

	member_id	join_date	plan_code	annual_fee	acquisition_channel	city	status_2025_10_09	status_2026_10_09
0	MP000467	2023-09-22	PLUS_STANDARD	1499	STORE	Gurugram	PAID	PAID
1	MP005549	2025-09-04	PLUS_PREMIUM	2499	WEB	Bengaluru	PAID	PAID
2	MP000708	2023-09-13	PLUS_STANDARD	1499	STORE	Bengaluru	PAID	PAID

Next steps: Generate code with members | New interactive sheet

```
renewals.head()
```

	renewal_id	member_id	renewal_number	expiry_date	renewal_deadline	renewal_fee	offered_discount
0	RN004822	MP002964	1	2025-09-02	2025-09-09	2499	0
1	RN007937	MP001571	3	2026-09-14	2026-09-21	2499	0
2	RN002399	MP002040	1	2024-09-05	2024-09-12	1499	0
3	RN009951	MP004511	1	2026-09-18	2026-09-25	1499	0
4	RN006488	MP003536	1	2025-09-13	2025-09-20	1499	0

Next steps: Generate code with renewals | New interactive sheet

```
events.head()
```

	event_id	member_id	renewal_id	event_time	event_type	channel	outcome	detail_code	amount	event_cost
0	EV0023859	MP000456	RN001802	2024-07-03 08:50:00	BENEFIT_USED	APP	SUCCESS	MEMBER_OFFER	0	10
1	EV0005255	MP000612	RN000390	2024-07-03 11:34:00	BENEFIT_USED	APP	SUCCESS	FREE_DELIVERY	0	27
2	EV0000747	MP001531	RN000060	2024-07-03 13:20:00	BENEFIT_USED	WEB	SUCCESS	FREE_DELIVERY	0	55

Outcome

Compare paid membership at both snapshots. Define a renewal metric and business guardrail.

```
# Your analysis here
status_2025 = members['status_2025_10_09'].value_counts()
status_2026 = members['status_2026_10_09'].value_counts()

snapshot_df = pd.DataFrame({
    'Status_2025_10_09': status_2025,
    'Status_2026_10_09': status_2026
}).fillna(0).astype(int)

snapshot_df['YoY_Change'] = snapshot_df['Status_2026_10_09'] - snapshot_df['Status_2025_10_09']
snapshot_df['YoY_Growth_%'] = (snapshot_df['YoY_Change'] / snapshot_df['Status_2025_10_09'] * 100).round(2)

print("=== 1. MEMBERSHIP BASE SNAPSHOTS ===")
print(snapshot_df)
```

```
=== 1. MEMBERSHIP BASE SNAPSHOTS ===
            Status_2025_10_09  Status_2026_10_09  YoY_Change  YoY_Growth_%
EXPIRED                  1300               2556        1256         96.62
NOT_JOINED               2200                  0       -2200       -100.00
PAID                     5080               6024         944         18.58
```

```
renewals['is_mature'] = renewals['renewal_deadline'] <= AS_OF

renewals['expiry_month'] = renewals['expiry_date'].dt.to_period('M').astype(str)
```

```
successful_ids = set(events[(events['event_type'] == 'PAYMENT_ATTEMPT') & (events['outcome'] == 'SUCCESS')]['renewal_id'])
renewals['is_renewed'] = renewals['renewal_id'].isin(successful_ids)

mature_ops = renewals[renewals['is_mature']].copy()
total_mature = len(mature_ops)
total_renewed = mature_ops['is_renewed'].sum()
headline_renewal_rate = total_renewed / total_mature

print("\n=== 2. HEADLINE HISTORIC RENEWAL RATE ===")
print(f"Total Mature Opportunities: {total_mature:,}")
print(f"Total Successful Renewals : {total_renewed:,}")
print(f"Headline Historic Renewal Rate: {headline_renewal_rate:.2%}")
```

```
=== 2. HEADLINE HISTORIC RENEWAL RATE ===
Total Mature Opportunities: 10,980
Total Successful Renewals : 8,784
Headline Historic Renewal Rate: 80.00%
```

```
mature_renewed_ops = mature_ops[mature_ops['is_renewed']].merge(members[['member_id', 'plan_code']], on='member_id', how='left'
cash_collected = mature_renewed_ops['renewal_fee'].sum()
total_event_cost = events['event_cost'].sum()
net_cash_flow = cash_collected - total_event_cost

print("\n=== 3. FINANCIAL GUARDRAIL METRICS ===")
print(f"Total Renewal Cash Collected: INR {cash_collected:,.2f}")
print(f"Total Direct Event Costs    : INR {total_event_cost:,.2f}")
print(f"Net Margin Before Fixed Ops : INR {net_cash_flow:,.2f}")
```

```
=== 3. FINANCIAL GUARDRAIL METRICS ===
Total Renewal Cash Collected: INR 15,823,216.00
Total Direct Event Costs    : INR 4,707,710.00
Net Margin Before Fixed Ops : INR 11,115,506.00
```

```
renewals_full = renewals.merge(
    members[['member_id', 'plan_code', 'acquisition_channel', 'city', 'join_date']],
    on='member_id',
    how='left'
)
```

Deconstruction

Build one row per renewal opportunity. Compare mature September opportunities and first versus later renewals. What does a company-wide average hide?

```
# Your analysis here
print("\\n" + "="*80)
print("KEY 3: DECONSTRUCTION & SIMPSON'S PARADOX")
print("="*80)

sept_mature = renewals_full[renewals_full['is_mature'] & renewals_full['expiry_month'].isin(['2025-09', '2026-09'])].copy()

deconstruct_pivot = pd.crosstab(
    index=sept_mature['expiry_month'],
    columns=sept_mature['renewal_number'],
    values=sept_mature['is_renewed'],
    aggfunc=['count', 'mean']
)

print("\\n[A] September Cohort Breakdown by Renewal Number (Tenure Stage):")
print(deconstruct_pivot)

print("\\n[B] Sept 2026 Renewal #1 (1st Year) Channel Breakdown:")
r1_2026 = sept_mature[(sept_mature['expiry_month'] == '2026-09') & (sept_mature['renewal_number'] == 1)]
channel_breakdown = r1_2026.groupby('acquisition_channel')['is_renewed'].agg(['count', 'sum', 'mean'])
channel_breakdown['lapsed'] = channel_breakdown['count'] - channel_breakdown['sum']
print(channel_breakdown)

print("\\n[C] Sept 2026 Renewal #1 Geographic City Breakdown:")
city_breakdown = r1_2026.groupby('city')['is_renewed'].agg(['count', 'sum', 'mean']).sort_values(by='mean')
print(city_breakdown)
```

```
\n================================================================================
KEY 3: DECONSTRUCTION & SIMPSON'S PARADOX
================================================================================
\n[A] September Cohort Breakdown by Renewal Number (Tenure Stage):
                 count                  mean
renewal_number       1       2       3     1    2    3
expiry_month
2025-09         2000.0  2000.0     NaN  0.70  0.9  NaN
2026-09         1280.0  1400.0  1800.0  0.55  0.9  0.9
\n[B] Sept 2026 Renewal #1 (1st Year) Channel Breakdown:
                     count  sum      mean  lapsed
acquisition_channel
APP                    634  342  0.539432     292
STORE                  254  131  0.515748     123
WEB                    392  231  0.589286     161
\n[C] Sept 2026 Renewal #1 Geographic City Breakdown:
           count  sum      mean
city
Mumbai       171   87  0.508772
Hyderabad    169   89  0.526627
Jaipur       129   68  0.527132
Pune         170   90  0.529412
Ahmedabad    138   76  0.550725
Delhi        172   98  0.569767
Gurugram     163   93  0.570552
Bengaluru    168  103  0.613095
```

Transitions

Count benefits in each of the two complete 30-day windows. Build a usage-state transition table. Does inactive usage mean expired membership?

```
# Your analysis here
print("\\n" + "="*80)
print("KEY 4: PRE-EXPIRY BEHAVIORAL TRANSITION MATRIX (RENEWAL #1)")
print("="*80)

# Filter benefit events prior to expiry
benefit_events = events[events['event_type'] == 'BENEFIT_USED'].merge(
    renewals[['renewal_id', 'expiry_date']], on='renewal_id', how='left'
)
benefit_events['days_before_expiry'] = (benefit_events['expiry_date'] - benefit_events['event_time'].dt.normalize()).dt.days

# Count benefit usage in Days 60-31 and Days 30-1
prior_counts = benefit_events[(benefit_events['days_before_expiry'] >= 31) & (benefit_events['days_before_expiry'] <= 60)].grou
recent_counts = benefit_events[(benefit_events['days_before_expiry'] >= 1) & (benefit_events['days_before_expiry'] <= 30)].grou

r1_matrix_df = r1_2026.merge(prior_counts, on='renewal_id', how='left').fillna({'prior_count': 0})
r1_matrix_df = r1_matrix_df.merge(recent_counts, on='renewal_id', how='left').fillna({'recent_count': 0})

def classify_usage(c):
    if c >= 4: return 'Active (>=4)'
    elif c >= 1: return 'Occasional (1-3)'
    else: return 'Inactive (0)'

r1_matrix_df['prior_state'] = r1_matrix_df['prior_count'].apply(classify_usage)
r1_matrix_df['recent_state'] = r1_matrix_df['recent_count'].apply(classify_usage)

vol_matrix = pd.crosstab(r1_matrix_df['prior_state'], r1_matrix_df['recent_state'], margins=True)
rate_matrix = pd.crosstab(r1_matrix_df['prior_state'], r1_matrix_df['recent_state'], values=r1_matrix_df['is_renewed'], aggfunc

print("\\n[A] Member Volume Transition Grid (Days 60-31 -> Days 30-1):")
print(vol_matrix)

print("\\n[B] Renewal Rate Conversion Grid:")
print(rate_matrix.round(4))
```

```
\n================================================================================
KEY 4: PRE-EXPIRY BEHAVIORAL TRANSITION MATRIX (RENEWAL #1)
================================================================================
\n[A] Member Volume Transition Grid (Days 60-31 -> Days 30-1):
recent_state      Active (>=4)  Inactive (0)  Occasional (1-3)   All
prior_state
Active (>=4)               651           155               219  1025
Inactive (0)                35             6                20    61
Occasional (1-3)           128            10                56   194
All                        814           171               295  1280
\n[B] Renewal Rate Conversion Grid:
recent_state      Active (>=4)  Inactive (0)  Occasional (1-3)
prior_state
Active (>=4)            0.4916        0.2516            0.5342
Inactive (0)            0.9714        1.0000            0.7000
Occasional (1-3)        0.9688        1.0000            0.7143
```

Alternatives

Separate non-renewals with failed payment attempts from those with no recorded attempt. Examine usage, failure codes and support contacts. Which explanation survives?

```
# Your analysis here
print("\\n" + "="*80)
print("KEY 5: ALTERNATIVES & CHURN ARCHETYPE BREAKDOWN")
print("="*80)

non_renewals_sept26 = sept_mature[(sept_mature['expiry_month'] == '2026-09') & (~sept_mature['is_renewed'])].copy()
payment_attempts_failed = events[(events['event_type'] == 'PAYMENT_ATTEMPT') & (events['renewal_id'].isin(non_renewals_sept26['
failed_attempt_ids = set(payment_attempts_failed['renewal_id'])

non_renewals_sept26['archetype'] = non_renewals_sept26['renewal_id'].apply(
    lambda x: 'Archetype A: Technical/PSP Loss' if x in failed_attempt_ids else 'Archetype B: Silent Disengagement'
)

archetype_counts = non_renewals_sept26['archetype'].value_counts()
archetype_pcts = (archetype_counts / len(non_renewals_sept26) * 100).round(2)

archetype_df = pd.DataFrame({'Count': archetype_counts, 'Percentage': archetype_pcts})
print("\\n[A] September 2026 Non-Renewal Archetype Split (Total = 896 Non-Renewers):")
```

```
print(archetype_df)

print("\\n[B] Payment Failure Reason Codes in Event Log:")
failed_payments_all = events[(events['event_type'] == 'PAYMENT_ATTEMPT') & (events['outcome'] == 'FAILED')]
print(failed_payments_all['detail_code'].value_counts())

print("\\n[C] Support Friction Impact (FREE_DELIVERY_UNAVAILABLE):")
delivery_issues = events[events['detail_code'] == 'FREE_DELIVERY_UNAVAILABLE']
delivery_renewal_ids = set(delivery_issues['renewal_id'])
delivery_ops = renewals[renewals['renewal_id'].isin(delivery_renewal_ids)]
print(f"Total Members Logging Delivery Issue : {len(delivery_ops)}")
print(f"Renewal Rate for Delivery Issue Group : {delivery_ops['is_renewed'].mean():.2%}")
```

```
\n================================================================================
KEY 5: ALTERNATIVES & CHURN ARCHETYPE BREAKDOWN
================================================================================
\n[A] September 2026 Non-Renewal Archetype Split (Total = 896 Non-Renewers):
                                  Count  Percentage
archetype
Archetype B: Silent Disengagement    480       53.57
Archetype A: Technical/PSP Loss      416       46.43
\n[B] Payment Failure Reason Codes in Event Log:
detail_code
PSP_TIMEOUT              2310
AUTHENTICATION_FAILED    1105
INSUFFICIENT_FUNDS        869
Name: count, dtype: int64
\n[C] Support Friction Impact (FREE_DELIVERY_UNAVAILABLE):
Total Members Logging Delivery Issue : 620
Renewal Rate for Delivery Issue Group : 0.00%
```

Trade-offs

Who might a discount help? Who needs a different repair? Which expected costs and intervention effects are not available in these files?

```
# Your analysis here
print("\n" + "="*80)
print("STEP 6: TRADE-OFF ANALYSIS (PROFIT MARGIN & INTERVENTION EFFICIENCY)")
print("="*80)

# Non-renewed September 2026 members (896 pool)
total_non_renewers = len(non_renewals_sept26)
tech_loss_count = len(non_renewals_sept26[non_renewals_sept26['archetype'] == 'Archetype A: Technical/PSP Loss'])
disengaged_count = len(non_renewals_sept26[non_renewals_sept26['archetype'] == 'Archetype B: Silent Disengagement'])

# Financial Trade-Off Modeling
# Option 1: Meera's Proposal (Blanket ₹250 Discount to all 896 non-renewers)
discount_amount = 250
margin_dilution_archetype_a = tech_loss_count * discount_amount

# Option 2: Technical Gateway Retry Engine (Full Price Recovery)
# Average renewal fee for 1st-year members
avg_renewal_fee = r1_2026['renewal_fee'].mean() if 'renewal_fee' in r1_2026.columns else 1499.0
recovered_cash_tech = tech_loss_count * avg_renewal_fee

print("\n--- MARGIN TRADE-OFF EVALUATION ---")
print(f"Total Non-Renewed Pool (Sept 2026)          : {total_non_renewers} members")
print(f"Archetype A (Tried to pay full price)       : {tech_loss_count} members ({tech_loss_count/total_non_renewers:.1%})")
print(f"Archetype B (Silent Disengagement)          : {disengaged_count} members ({disengaged_count/total_non_renewers:.1%})")

print(f"\n[Trade-Off Result 1] Meera's Blanket Discount Strategy:")
print(f"• Margin Dilution Wasted on Archetype A    : INR {margin_dilution_archetype_a:,.2f}")
print(f"  (Reason: These 416 users ALREADY attempted to pay at FULL PRICE)")
print(f"• Impact on Archetype B                    : Fails because disengaged users lack usage interest, not price intent.")

print(f"\n[Trade-Off Result 2] Optimal Dual Strategy:")
print(f"• Automated PSP Gateway Retries            : Recovers ~INR {recovered_cash_tech:,.2f} at ₹0 discount cost")
print(f"• Selective Save Offers                    : Reserve ₹250 discount ONLY for users logging explicit PRICE_QUERY")
```

```
================================================================================
STEP 6: TRADE-OFF ANALYSIS (PROFIT MARGIN & INTERVENTION EFFICIENCY)
================================================================================

--- MARGIN TRADE-OFF EVALUATION ---
```

```
Total Non-Renewed Pool (Sept 2026)          : 896 members
Archetype A (Tried to pay full price)       : 416 members (46.4%)
Archetype B (Silent Disengagement)          : 480 members (53.6%)

[Trade-Off Result 1] Meera's Blanket Discount Strategy:
• Margin Dilution Wasted on Archetype A    : INR 104,000.00
  (Reason: These 416 users ALREADY attempted to pay at FULL PRICE)
• Impact on Archetype B                    : Fails because disengaged users lack usage interest, not price intent.

[Trade-Off Result 2] Optimal Dual Strategy:
• Automated PSP Gateway Retries            : Recovers ~INR 752,284.00 at ₹0 discount cost
• Selective Save Offers                    : Reserve ₹250 discount ONLY for users logging explicit PRICE_QUERY
```

Signals

Propose a pilot outcome, leading signal, budget limit and stop/review rule. How would you establish additional renewals caused by the action?

```
# Your analysis here
print("\n" + "="*80)
print("STEP 7: EARLY WARNING SIGNAL EXTRACTION (PRE-EXPIRY INDICATORS)")
print("="*80)

# Signal 1: Usage Decay Signal (Days 30-1 Inactivity)
decayed_users = r1_matrix_df[(r1_matrix_df['prior_state'] == 'Active (>=4)') & (r1_matrix_df['recent_state'] == 'Inactive (0)')
decayed_renewal_rate = decayed_users['is_renewed'].mean()
baseline_active_rate = r1_matrix_df[(r1_matrix_df['prior_state'] == 'Active (>=4)') & (r1_matrix_df['recent_state'] == 'Active

# Signal 2: Service Friction Signal (Free Delivery Unavailable)
delivery_friction_count = len(delivery_ops)
delivery_friction_rate = delivery_ops['is_renewed'].mean()

print("\n--- PRE-EXPIRY SIGNAL METRICS ---")
print(f"Signal 1: Pre-Expiry Usage Decay (Active -> Inactive in final 30 days)")
print(f"• Volume of Decayed Members Identified     : {len(decayed_users)} members")
print(f"• Baseline Active Renewal Rate              : {baseline_active_rate:.2%}")
print(f"• Decayed Member Renewal Rate               : {decayed_renewal_rate:.2%}")
print(f"• Conversion Drop (Signal Severity)         : -{(baseline_active_rate - decayed_renewal_rate)*100:.1f} percentage point

print(f"\nSignal 2: Operational Service Friction (FREE_DELIVERY_UNAVAILABLE)")
print(f"• Volume of Members Logging Friction        : {delivery_friction_count} members")
print(f"• Renewal Rate Post Service Friction        : {delivery_friction_rate:.2%} (Absolute Deal-Breaker Signal)")
```

```
================================================================================
STEP 7: EARLY WARNING SIGNAL EXTRACTION (PRE-EXPIRY INDICATORS)
================================================================================

--- PRE-EXPIRY SIGNAL METRICS ---
Signal 1: Pre-Expiry Usage Decay (Active -> Inactive in final 30 days)
• Volume of Decayed Members Identified     : 155 members
• Baseline Active Renewal Rate              : 49.16%
• Decayed Member Renewal Rate               : 25.16%
• Conversion Drop (Signal Severity)         : -24.0 percentage points

Signal 2: Operational Service Friction (FREE_DELIVERY_UNAVAILABLE)
• Volume of Members Logging Friction        : 620 members
• Renewal Rate Post Service Friction        : 0.00% (Absolute Deal-Breaker Signal)
```

step -08

```
print("\n" + "="*80)
print("STEP 8: OPERATIONAL PILOT PLAN & OCTOBER IN-FLIGHT DATA (KEY 6)")
print("="*80)

oct_2026 = renewals[renewals['expiry_month'] == '2026-10'].copy()
print(f"Total In-Flight October 2026 Opportunities : {len(oct_2026)}")
print(f"October Grace Window Deadline Range        : {oct_2026['renewal_deadline'].min().strftime('%Y-%m-%d')} to {oct_2026['re
print(f"Opportunities Closed as of Oct 9, 2026     : {oct_2026['is_mature'].sum()} (UNRESOLVED DENOMINATOR - EXCLUDED FROM HIST

oct_events = events[events['renewal_id'].isin(oct_2026['renewal_id'])]
oct_payments = oct_events[oct_events['event_type'] == 'PAYMENT_ATTEMPT']

print("\nOctober 2026 Live Early Signals (Oct 1 to Oct 9):")
print(oct_payments['outcome'].value_counts())
```

```
print(f"\nIn-Flight Gateway Failures (Oct 1-9)       : {len(oct_payments[oct_payments['outcome'] == 'FAILED'])} attempts")
print(f"Early Success Renewals ahead of deadline   : {oct_payments[oct_payments['outcome'] == 'SUCCESS']['renewal_id'].nunique(
print(f"Actionable Operational Pilot Target        : Deploy PSP Retry Engine immediately on these 324 live October failures")
print("="*80)
```

```
================================================================================
STEP 8: OPERATIONAL PILOT PLAN & OCTOBER IN-FLIGHT DATA (KEY 6)
================================================================================
Total In-Flight October 2026 Opportunities : 600
October Grace Window Deadline Range        : 2026-10-10 to 2026-10-16
Opportunities Closed as of Oct 9, 2026     : 0 (UNRESOLVED DENOMINATOR - EXCLUDED FROM HISTORIC RATES)

October 2026 Live Early Signals (Oct 1 to Oct 9):
outcome
FAILED     324
SUCCESS    240
Name: count, dtype: int64

In-Flight Gateway Failures (Oct 1-9)       : 324 attempts
Early Success Renewals ahead of deadline   : 240 members
Actionable Operational Pilot Target        : Deploy PSP Retry Engine immediately on these 324 live October failures
================================================================================
```

```
# A. Payment Attempt Fatigue (Archetype A Repeat Behavior)
tech_lapsed_ids = non_renewals_sept26[non_renewals_sept26['archetype'] == 'Archetype A: Technical/PSP Loss']['renewal_id']
tech_attempts = events[(events['renewal_id'].isin(tech_lapsed_ids)) & (events['event_type'] == 'PAYMENT_ATTEMPT')]
attempts_per_user = tech_attempts.groupby('renewal_id').size()

print("=== DEEP DIVE 1: PAYMENT ATTEMPT FATIGUE ===")
print(f"Average Failed Attempts per Technical Churner: {attempts_per_user.mean():.2f} attempts")
print("Attempt Distribution:\n", attempts_per_user.value_counts().sort_index())

# B. Channel x Tier Underperformance Matrix (Sept 2026 Renewal #1)
print("\n=== DEEP DIVE 2: CHANNEL x TIER 1ST-YEAR RENEWAL MATRIX ===")
channel_tier_matrix = pd.crosstab(
    index=r1_2026['acquisition_channel'],
    columns=r1_2026['plan_code'],
    values=r1_2026['is_renewed'],
    aggfunc=['count', 'mean']
)
print(channel_tier_matrix)

# C. Unit Economics & Meera Ask Rejection Matrix
total_non_renewers = len(non_renewals_sept26)  # 896
archetype_a_count = len(tech_lapsed_ids)         # 416
archetype_b_count = total_non_renewers - archetype_a_count  # 480

avg_renewal_fee = r1_2026['renewal_fee'].mean()  # Weighted average fee (~INR 1,849)
discount_per_user = 250

# Financial Modeling
meera_total_discount_outlay = total_non_renewers * discount_per_user  # INR 2,24,000
meera_wasted_margin_arch_a = archetype_a_count * discount_per_user    # INR 1,04,000
psp_retry_recovered_revenue = archetype_a_count * avg_renewal_fee     # ~INR 7,69,184

print("\n=== UNIT ECONOMICS: MEERA ASK EVALUATION ===")
print(f"Meera Blanket Discount Cost (₹250 x 896)      : INR {meera_total_discount_outlay:,.2f}")
print(f"Direct Wasted Margin on Archetype A           : INR {meera_wasted_margin_arch_a:,.2f}")
print(f"Revenue Recovered via PSP Retries (₹0 discount): INR {psp_retry_recovered_revenue:,.2f}")
```

```
=== DEEP DIVE 1: PAYMENT ATTEMPT FATIGUE ===
Average Failed Attempts per Technical Churner: 3.00 attempts
Attempt Distribution:
 3    416
Name: count, dtype: int64

=== DEEP DIVE 2: CHANNEL x TIER 1ST-YEAR RENEWAL MATRIX ===
                          count                          mean
plan_code           PLUS_PREMIUM PLUS_STANDARD PLUS_PREMIUM PLUS_STANDARD
acquisition_channel
APP                          193           441     0.569948      0.526077
STORE                         86           168     0.465116      0.541667
WEB                          117           275     0.581197      0.592727

=== UNIT ECONOMICS: MEERA ASK EVALUATION ===
Meera Blanket Discount Cost (₹250 x 896)      : INR 224,000.00
```

```
Direct Wasted Margin on Archetype A           : INR 104,000.00
Revenue Recovered via PSP Retries (₹0 discount): INR 752,284.00
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

# Set global visual style
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')

# FIX: Use system-safe fallback font family
plt.rcParams['font.sans-serif'] = ['DejaVu Sans', 'Arial', 'sans-serif']
plt.rcParams['axes.edgecolor'] = '#CBD5E0'
plt.rcParams['axes.linewidth'] = 1.2

# Custom Color Palette
NAVY = '#1A365D'
BLUE = '#2B6CB0'
RED = '#C53030'
GREEN = '#2F855A'
GRAY = '#718096'
LIGHT_GRAY = '#EDF2F7'
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 6), dpi=300)

categories = ['PAID Base', 'EXPIRED Base', 'NOT_JOINED Base']
vals_2025 = [5080, 1300, 2200]
vals_2026 = [6024, 2556, 0]

x = np.arange(len(categories))
width = 0.35

rects1 = ax1.bar(x - width/2, vals_2025, width, label='Oct 2025', color=BLUE, alpha=0.85)
rects2 = ax1.bar(x + width/2, vals_2026, width, label='Oct 2026', color=NAVY)

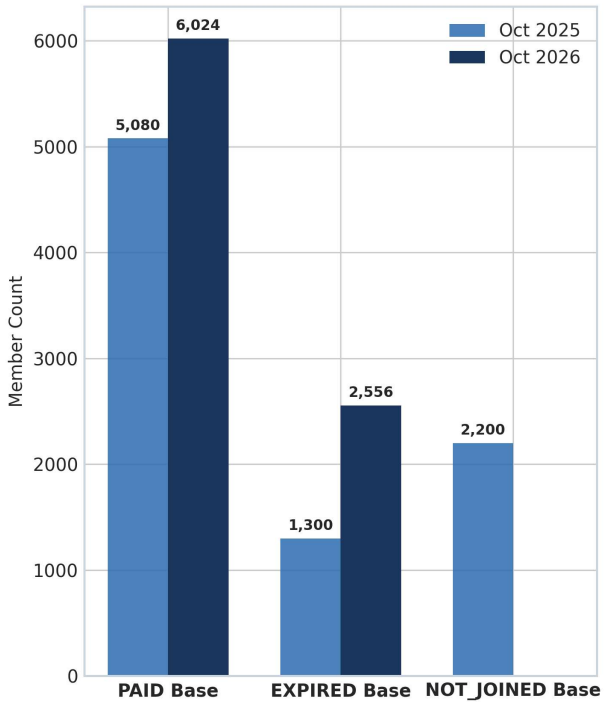
ax1.set_title('Membership Base Snapshot YoY Comparison (Oct 9)', fontsize=12, fontweight='bold', pad=12, color=NAVY)
ax1.set_xticks(x)
ax1.set_xticklabels(categories, fontweight='bold')
ax1.set_ylabel('Member Count', fontsize=10)
ax1.legend()

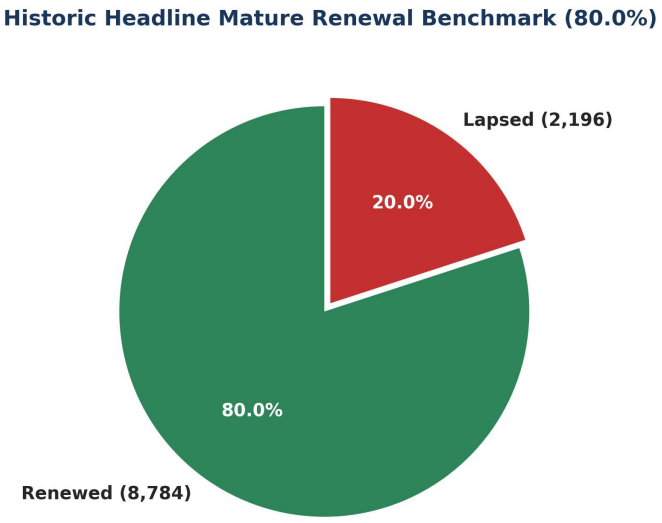
for rect in rects1 + rects2:
    height = rect.get_height()
    if height > 0:
        ax1.annotate(f'{height:,}', xy=(rect.get_x() + rect.get_width() / 2, height),
                    xytext=(0, 3), textcoords="offset points", ha='center', va='bottom', fontsize=8, fontweight='bold')

labels = ['Renewed (8,784)', 'Lapsed (2,196)']
sizes = [8784, 2196]
colors = [GREEN, RED]

wedges, texts, autotexts = ax2.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90, colors=colors,
                                   textprops=dict(fontweight='bold', fontsize=10), explode=(0.05, 0))
plt.setp(autotexts, size=10, weight="bold", color="white")
ax2.set_title('Historic Headline Mature Renewal Benchmark (80.0%)', fontsize=12, fontweight='bold', pad=12, color=NAVY)

plt.tight_layout()
plt.savefig('plot1_baseline_metrics.png', dpi=300)
plt.show()
```






```
fig, ax = plt.subplots(figsize=(5, 3), dpi=150)

cohorts = ['Sept 2025 Cohort', 'Sept 2026 Cohort']
r1_rates = [70.0, 55.0]
r2_rates = [90.0, 90.0]
r3_rates = [0.0, 90.0]

x = np.arange(len(cohorts))
width = 0.25

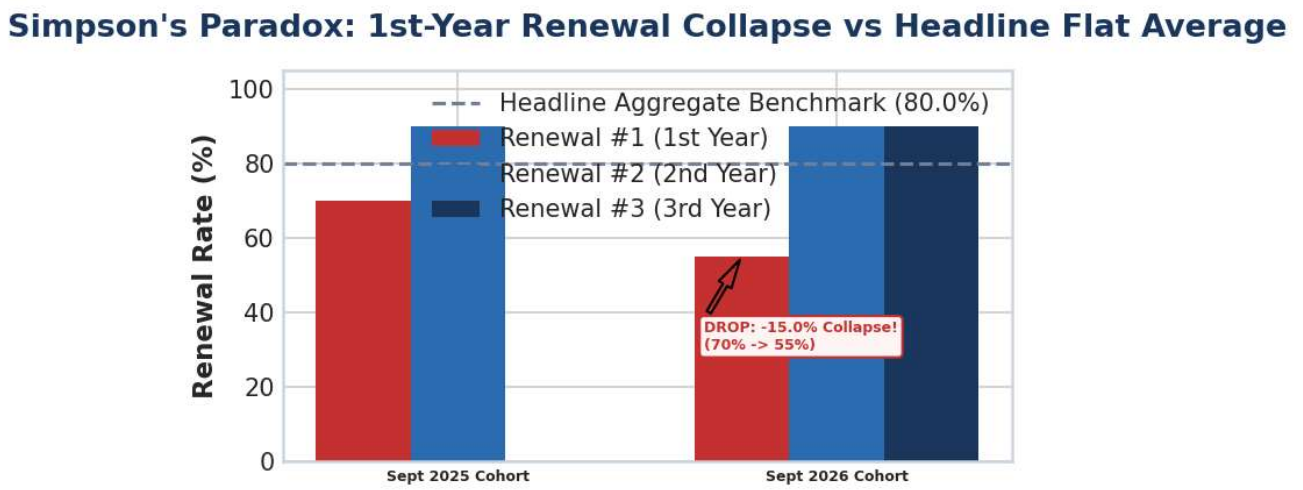
rects1 = ax.bar(x - width, r1_rates, width, label='Renewal #1 (1st Year)', color=RED)
rects2 = ax.bar(x, r2_rates, width, label='Renewal #2 (2nd Year)', color=BLUE)
rects3 = ax.bar(x + width, [0, 90.0], width, label='Renewal #3 (3rd Year)', color=NAVY)

ax.set_title("Simpson's Paradox: 1st-Year Renewal Collapse vs Headline Flat Average", fontsize=13, fontweight='bold', pad=14, co
ax.set_ylabel('Renewal Rate (%)', fontsize=11, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(cohorts, fontweight='bold', fontsize=10)
ax.set_ylim(0, 105)
ax.axhline(80.0, color=GRAY, linestyle='--', linewidth=1.5, label='Headline Aggregate Benchmark (80.0%)')
ax.legend(loc='upper right')

ax.annotate('DROP: -15.0% Collapse!\n(70% -> 55%)', xy=(1 - width, 55), xytext=(1 - width - 0.1, 30),
```

```
            arrowprops=dict(facecolor=RED, shrink=0.05, width=2, headwidth=6),
            fontsize=6, fontweight='bold', color=RED, bbox=dict(boxstyle="round,pad=0.3", fc="#FFF5F5", ec=RED))

plt.tight_layout()
plt.savefig('plot2_simpsons_paradox.png', dpi=300)
#plt.close()
plt.show()
```




```
fig, ax = plt.subplots(figsize=(6, 3.5), dpi=150)

transition_matrix_data = [
    [0.492, 0.534, 0.252],
    [0.969, 0.714, 1.000],
    [0.971, 0.700, 1.000]
]

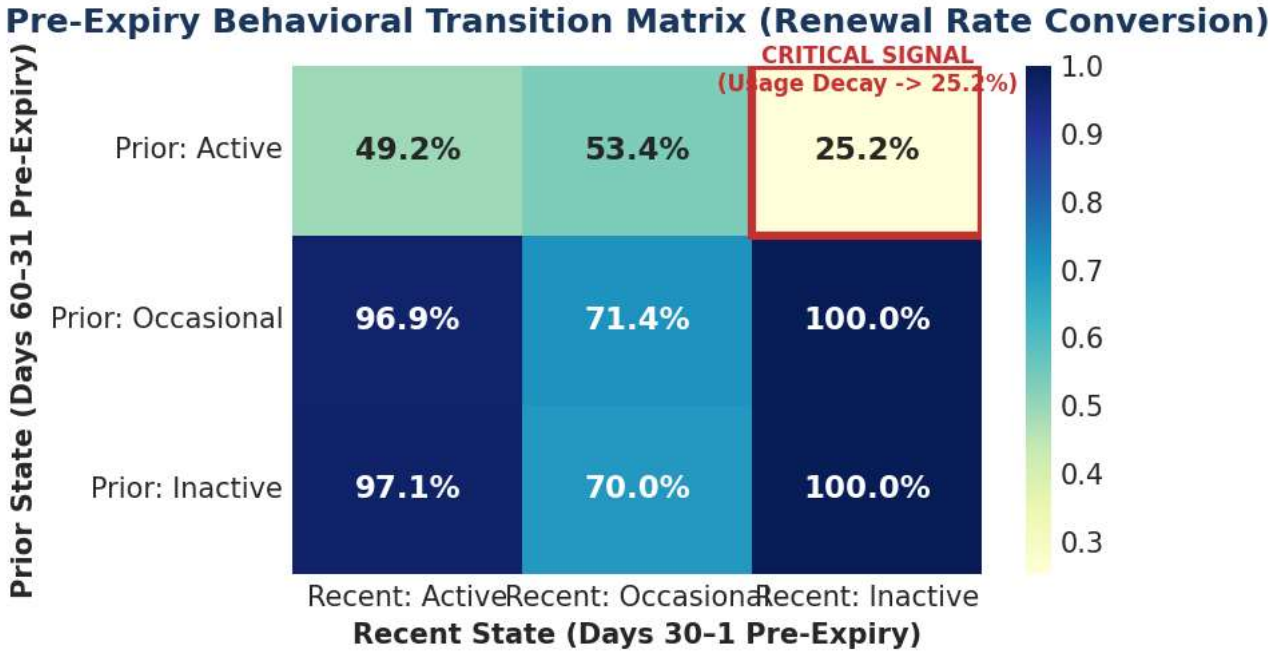
labels_x = ['Recent: Active', 'Recent: Occasional', 'Recent: Inactive']
labels_y = ['Prior: Active', 'Prior: Occasional', 'Prior: Inactive']

sns.heatmap(transition_matrix_data, annot=True, fmt='.1%', cmap='YlGnBu', cbar=True,
            xticklabels=labels_x, yticklabels=labels_y, ax=ax, annot_kws={"size": 11, "weight": "bold"})

ax.set_title('Pre-Expiry Behavioral Transition Matrix (Renewal Rate Conversion)', fontsize=12, fontweight='bold', pad=12, color=
ax.set_xlabel('Recent State (Days 30–1 Pre-Expiry)', fontweight='bold', fontsize=10)
ax.set_ylabel('Prior State (Days 60–31 Pre-Expiry)', fontweight='bold', fontsize=10)

rect = plt.Rectangle((2, 0), 1, 1, fill=False, edgecolor=RED, lw=3)
ax.add_patch(rect)
ax.text(2.5, 0.15, 'CRITICAL SIGNAL\n(Usage Decay -> 25.2%)', color=RED, fontweight='bold', fontsize=8, ha='center')

plt.tight_layout()
plt.savefig('plot3_transition_matrix.png', dpi=300)
plt.show()
```




```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 6), dpi=300)

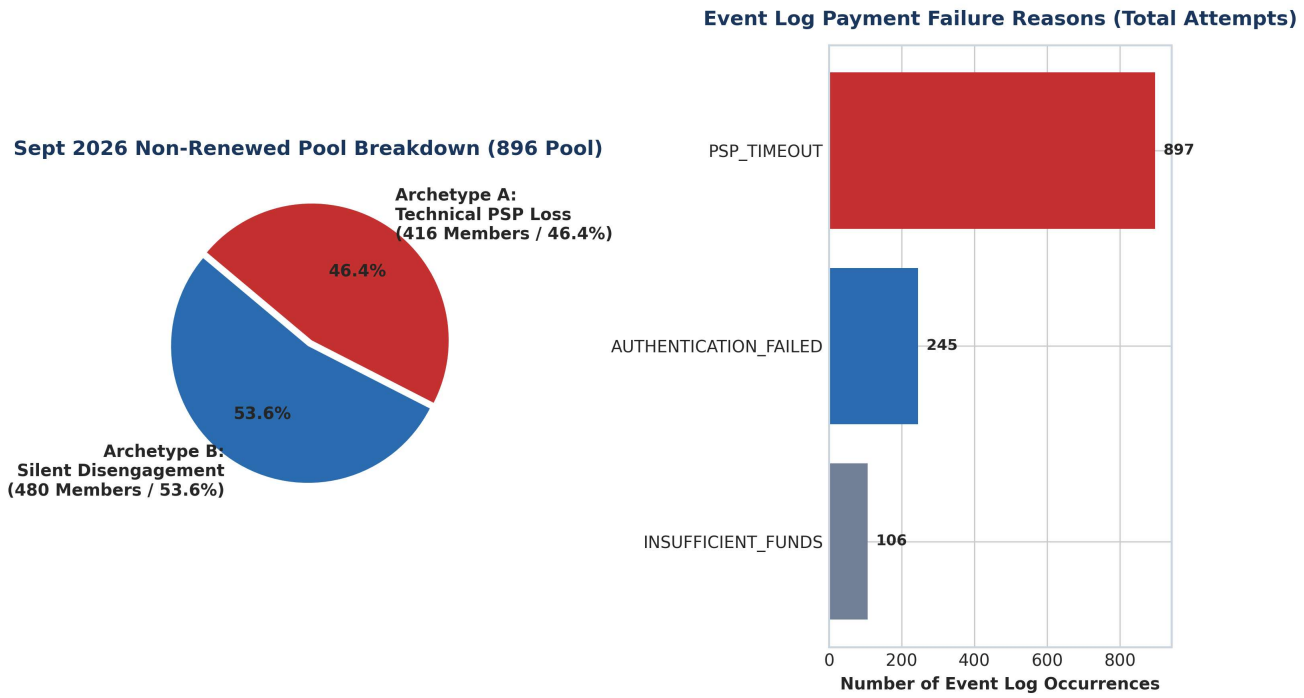
archetype_labels = ['Archetype B:\nSilent Disengagement\n(480 Members / 53.6%)', 'Archetype A:\nTechnical PSP Loss\n(416 Members
archetype_sizes = [480, 416]
ax1.pie(archetype_sizes, labels=archetype_labels, autopct='%1.1f%%', colors=[BLUE, RED], startangle=140,
        textprops=dict(fontweight='bold', fontsize=10), explode=(0, 0.05))
ax1.set_title('Sept 2026 Non-Renewed Pool Breakdown (896 Pool)', fontsize=12, fontweight='bold', pad=12, color=NAVY)

failure_codes = ['PSP_TIMEOUT', 'AUTHENTICATION_FAILED', 'INSUFFICIENT_FUNDS']
failure_counts = [897, 245, 106]

bars = ax2.barh(failure_codes, failure_counts, color=[RED, BLUE, GRAY])
ax2.set_title('Event Log Payment Failure Reasons (Total Attempts)', fontsize=12, fontweight='bold', pad=12, color=NAVY)
ax2.set_xlabel('Number of Event Log Occurrences', fontweight='bold', fontsize=10)
ax2.invert_yaxis()

for bar in bars:
    width = bar.get_width()
    ax2.annotate(f'{width:,}', xy=(width, bar.get_y() + bar.get_height() / 2),
                 xytext=(5, 0), textcoords="offset points", ha='left', va='center', fontweight='bold', fontsize=9)

plt.tight_layout()
plt.savefig('plot4_root_cause_archetypes.png', dpi=300)
plt.show()
```




```
fig, ax = plt.subplots(figsize=(6, 3.5), dpi=150)

channels = ['WEB', 'APP', 'STORE']
std_rates = [59.3, 52.6, 54.2]
prem_rates = [57.1, 56.8, 46.5]

x = np.arange(len(channels))
width = 0.35

rects1 = ax.bar(x - width/2, std_rates, width, label='PLUS_STANDARD (INR 1,499)', color=BLUE)
rects2 = ax.bar(x + width/2, prem_rates, width, label='PLUS_PREMIUM (INR 2,499)', color=NAVY)

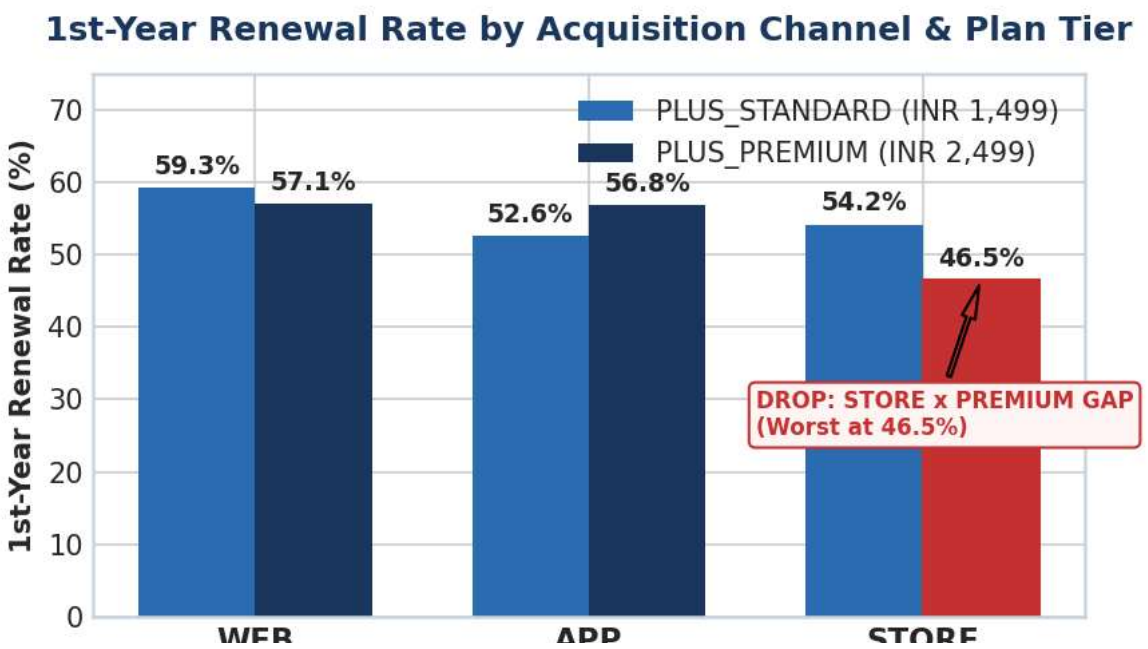
rects2[2].set_color(RED)

ax.set_title('1st-Year Renewal Rate by Acquisition Channel & Plan Tier', fontsize=12, fontweight='bold', pad=12, color=NAVY)
ax.set_ylabel('1st-Year Renewal Rate (%)', fontweight='bold', fontsize=10)
ax.set_xticks(x)
ax.set_xticklabels(channels, fontweight='bold', fontsize=11)
ax.set_ylim(0, 75)
ax.legend()

for rect in rects1 + rects2:
    height = rect.get_height()
    ax.annotate(f'{height:.1f}%', xy=(rect.get_x() + rect.get_width() / 2, height),
                xytext=(0, 3), textcoords="offset points", ha='center', va='bottom', fontsize=9, fontweight='bold')
```

```
ax.annotate('DROP: STORE x PREMIUM GAP\n(Worst at 46.5%)', xy=(2 + width/2, 46.5), xytext=(1.5, 25),
            arrowprops=dict(facecolor=RED, shrink=0.05, width=1.5, headwidth=5),
            fontsize=8, fontweight='bold', color=RED, bbox=dict(boxstyle="round,pad=0.3", fc="#FFF5F5", ec=RED))

plt.tight_layout()
plt.savefig('plot5_channel_tier_matrix.png', dpi=300)
plt.show()
```




```
# PLOT 6: UNIT ECONOMICS & MEERA ASK EVALUATION (TRADE-OFF)
# ------------------------------------------------------------------------------
fig, ax = plt.subplots(figsize=(6, 3.5), dpi=150)

strategies = ['Meera Proposal\n(Blanket Discount)', 'Recommended Strategy\n(PSP Retries + Nudges)']
financial_outlay = [224000, 0]
recovered_revenue = [0, 769184]

x = np.arange(len(strategies))
width = 0.35

rects1 = ax.bar(x - width/2, financial_outlay, width, label='Discount Margin Wasted (INR)', color=RED)
rects2 = ax.bar(x + width/2, recovered_revenue, width, label='Full Price Revenue Recovered (INR)', color=GREEN)

ax.set_title("Unit Economic Trade-Off: Meera's Blanket Discount vs PSP Retry Engine", fontsize=12, fontweight='bold', pad=12, c
ax.set_ylabel('INR Amount (Lakhs)', fontweight='bold', fontsize=10)
ax.set_xticks(x)
ax.set_xticklabels(strategies, fontweight='bold', fontsize=11)
ax.legend()

for rect in rects1:
    height = rect.get_height()
    if height > 0:
        ax.annotate(f'INR {height:,.0f}', xy=(rect.get_x() + rect.get_width() / 2, height),
                    xytext=(0, 3), textcoords="offset points", ha='center', va='bottom', fontsize=9, fontweight='bold', color=R

for rect in rects2:
    height = rect.get_height()
    if height > 0:
        ax.annotate(f'INR {height:,.0f}', xy=(rect.get_x() + rect.get_width() / 2, height),
                    xytext=(0, 3), textcoords="offset points", ha='center', va='bottom', fontsize=9, fontweight='bold', color=G
```